

# AWQ: Protect the Salient Weights, Worked Out by Hand

NOT ALL WEIGHTS ARE EQUAL — SCALE THE IMPORTANT  $\sim 1\%$  UP BEFORE YOU ROUND

**What this document does.** It works Activation-aware Weight Quantization on a layer with four input channels, one of which is *salient* (its activations are  $50\times$  larger than the rest). We show, by hand, that naive INT4 puts 98% of its output error on that one channel, then use the AWQ identity  $(W \cdot s)(X/s) = WX$  to scale the salient weight *up* before rounding and divide the activation *down* to keep the output identical. The activation-weighted error drops from 0.2487 to 0.0259 — an 89.6% reduction. Every figure is reproduced in Appendix A (`m02_t03_awq_engine.py`).

**The one idea.** A rounding error only matters in proportion to the activation it multiplies. A tiny weight error on a channel whose activations are huge produces a huge output error; the same weight error on a quiet channel is invisible. So weights are *not* uniformly important. AWQ finds the  $\sim 1\%$  of channels with large activations and spends extra grid resolution on their weights — for free, because scaling a weight up and its activation down leaves the math untouched.

CONCEPTS COVERED, IN ORDER

activation-weighted error and why it is the right target; the salient-channel observation; the algebraic identity  $(W \cdot s)(X/s) = WX$ ; per-channel scale  $s_j$  and the grid-search over  $\alpha$  in  $s_j = \max |X_j|^\alpha / \max |W_j|^{1-\alpha}$ ; a full worked example where protecting one channel collapses the error; how AWQ compares to GPTQ.

INTERNAL LEARNING MATERIAL. NUMBERS ARE REPRODUCIBLE FROM APPENDIX A. v1.0

## Contents

|     |                                                 |   |
|-----|-------------------------------------------------|---|
| 1   | The object under study                          | 2 |
| 2   | Step 1 — The right error to minimize            | 2 |
| 3   | Step 2 — The baseline, and why it fails         | 2 |
| 4   | Step 3 — The AWQ identity                       | 3 |
| 5   | Step 4 — Grid-search the salient scale, by hand | 3 |
| 6   | Step 5 — AWQ in context                         | 4 |
| 7   | The argument, at a glance                       | 4 |
| 7.1 | Equation cheat-sheet . . . . .                  | 5 |

## Appendix A — Reference implementation

5

### 1 The object under study

One output row of a linear layer, four input channels. Channel 0 is salient: a calibration pass shows its mean activation magnitude is  $50\times$  the others.

| Quantity                      | Symbol           | Value here                       |
|-------------------------------|------------------|----------------------------------|
| Weight row                    | $W$              | [0.18, 0.92, -0.55, 0.30]        |
| Mean  activation  per channel | $x_{\text{mag}}$ | [5.00, 0.10, 0.12, 0.08]         |
| Salient channel               | —                | channel 0 ( $50\times$ the rest) |
| Target width                  | $b$              | 4 bits, symmetric per-tensor     |

#### Intuition

Think of the layer’s output as a choir. Each weight is a singer and its activation is how loud that singer’s microphone is turned up. A singer slightly off-key (a rounding error) ruins the performance only if their mic is loud. AWQ identifies the loud mics (salient channels) and makes sure those singers are tuned most precisely, even if it means spending a little less care on the near-silent ones. Tuning the muted singers perfectly is wasted effort.

### 2 Step 1 — The right error to minimize

Plain quantization minimizes weight error  $|W - \hat{W}|$ . But the layer’s output is  $WX$ , so what actually reaches the next layer is the weight error *scaled by the activation*:

Activation-weighted error — what AWQ targets

$$E = \sum_j |W_j - \hat{W}_j| \cdot \max |X_j|.$$

A weight error on a channel with big activations is amplified; on a quiet channel it is damped. This single reweighting is the whole difference between AWQ and round-to-nearest.

### 3 Step 2 — The baseline, and why it fails

The symmetric per-tensor INT4 scale is set by the largest weight,  $\max |W| = 0.92$ :

$$s_q = \frac{0.92}{7} = 0.13143.$$

Now quantize each weight to the nearest multiple of  $s_q$ :

| $W_j$ | $\hat{W}_j$ | $ W_j - \hat{W}_j $ | $\max  X_j $ | contribution to $E$ |
|-------|-------------|---------------------|--------------|---------------------|
| 0.18  | 0.1314      | 0.0486              | 5.00         | <b>0.2429</b>       |
| 0.92  | 0.9200      | 0.0000              | 0.10         | 0.0000              |
| -0.55 | -0.5257     | 0.0243              | 0.12         | 0.0029              |
| 0.30  | 0.2629      | 0.0371              | 0.08         | 0.0030              |

Total  $E = 0.2487$ . **The salient channel alone is 98% of it.** The cruel irony: the salient weight 0.18 is *small*, so the per-tensor grid (set by the far-away 0.92) gives it a coarse share — exactly the weight that can least afford a coarse grid gets the coarsest one.

#### Watch out

This is why per-tensor RTN loses 2–3 accuracy points at W4 while AWQ loses  $<0.5$ : the salient channels are where the damage concentrates, and the naive scheme protects them *least*. Run a calibration pass on any trained LLM and you will see  $\sim 1\%$  of input channels with activation magnitudes 10–100 $\times$  the rest — a real, observable structure.

## 4 Step 3 — The AWQ identity

The fix uses one trivial algebraic fact. Scale a weight channel *up* by  $s > 1$  and the matching activation channel *down* by the same  $s$ :

The scaling identity — output is unchanged

$$(W_j \cdot s) (X_j / s) = W_j X_j.$$

The product is identical, so the layer computes the same thing. But the *quantization* of  $W_j \cdot s$  is different: an inflated weight is farther from zero, lands on a finer relative grid, and suffers less rounding error. The activation's  $/s$  is folded into the previous layer's output scale at no runtime cost — “scale, quantize, bake.”

#### Why inflation protects a weight

On a fixed grid of spacing  $s_q$ , the *relative* rounding error of a value  $v$  is about  $s_q/(2|v|)$ . Doubling  $|v|$  halves the relative error. By scaling the salient weight up, AWQ moves it to a magnitude where the same absolute grid step is a smaller fraction of the value — then divides the activation back down so the output is untouched. Protection without cost.

## 5 Step 4 — Grid-search the salient scale, by hand

We sweep a scale  $s$  applied to the salient channel's weight, requantize the row, divide  $\hat{W}_0$  back by  $s$ , and recompute  $E$ :

| $s$ | $W_0 \cdot s$ | recovered $\hat{W}_0 = \hat{q}_0/s$ | error $E$     |
|-----|---------------|-------------------------------------|---------------|
| 1.0 | 0.180         | 0.1314                              | 0.2487        |
| 1.5 | 0.270         | 0.1752                              | 0.0297        |
| 2.0 | 0.360         | 0.1971                              | 0.0916        |
| 3.0 | 0.540         | 0.1752                              | 0.0297        |
| 5.0 | 0.900         | 0.1840                              | <b>0.0259</b> |

#### Why $s=5$ wins

At  $s = 5$ ,  $W_0 \cdot s = 0.90$ , which is almost exactly  $7 s_q = 0.92$  — so it snaps to code 7 with tiny residual. Dividing back:  $\hat{W}_0 = 7 s_q / 5 = 0.184$ , versus the true 0.18. The salient-channel error collapses from  $|0.18 - 0.1314| = 0.0486$  to  $|0.18 - 0.184| = 0.0040$  — a 12 $\times$

improvement on the channel that dominated the loss.

$$E : 0.2487 \rightarrow 0.0259, \quad \text{a 89.6\% reduction.} \quad \checkmark$$

#### Mini-example — the production scale formula

At scale, AWQ does not search one channel at a time; it sets a per-channel scale from both activation and weight magnitude, tuned by a single migration strength  $\alpha$ :

$$s_j = \frac{\max |X_j|^\alpha}{\max |W_j|^{1-\alpha}}.$$

For our layer (normalized so  $\bar{s} = 1$ ): at  $\alpha=0.5$  the salient channel gets  $s_0 = 3.20$ ; at  $\alpha=1.0$ ,  $s_0 = 3.77$ .  $\alpha=0$  ignores activations (weight-only,  $s_0=1.88$ );  $\alpha=1$  scales purely by activation. A grid of  $\sim 20$  values of  $\alpha$ , under a second per layer, picks the best — usually near  $\alpha=0.5$ .

## 6 Step 5 — AWQ in context

| Method (Llama-3-70B, W4 $g=128$ ) | MMLU | $\Delta$ from FP16 |
|-----------------------------------|------|--------------------|
| FP16 baseline                     | 79.5 | —                  |
| RTN W4                            | 76.1 | −3.4               |
| AWQ W4                            | 79.2 | −0.3               |
| GPTQ W4                           | 79.0 | −0.5               |

#### AWQ vs. GPTQ — two cures for the same disease

GPTQ (Topic 2) *compensates* rounding error by nudging other weights via the Hessian. AWQ *prevents* it on the channels that matter by scaling them up first. AWQ matches GPTQ on accuracy but is simpler: no Hessian inversion, pure symmetric quantization (no per-group zero-points), and  $\sim 30$  minutes calibration for 70B versus GPTQ’s 1–3 hours. That simplicity — and the fact the activation scale folds into the prior layer at zero runtime cost — is why **AWQ is vLLM’s default** for W4A16 and composes cleanly with INT8/FP8 activation quantization (Topic 4).

## 7 The argument, at a glance

| # | Step           | Result                                                            |
|---|----------------|-------------------------------------------------------------------|
| 1 | Right target   | $E = \sum_j  W_j - \hat{W}_j  \max  X_j $ (activation-weighted)   |
| 2 | Baseline fails | RTN W4 $E = 0.2487$ ; salient channel = 98%                       |
| 3 | Identity       | $(W \cdot s)(X/s) = WX$ ; inflate weight $\Rightarrow$ finer grid |
| 4 | Worked search  | best $s=5 \Rightarrow E = 0.0259$ (−89.6%)                        |
| 5 | In context     | AWQ −0.3 MMLU, simpler than GPTQ, vLLM default                    |

## 7.1 Equation cheat-sheet

|                           |                                                   |
|---------------------------|---------------------------------------------------|
| Activation-weighted error | $E = \sum_j  W_j - \hat{W}_j  \max  X_j $         |
| Scaling identity          | $(W_j \cdot s)(X_j/s) = W_j X_j$                  |
| Per-channel scale         | $s_j = \max  X_j ^\alpha / \max  W_j ^{1-\alpha}$ |
| Normalization             | $s \leftarrow s/\bar{s}$                          |
| Recover weight            | $\hat{W}_j = q(W_j \cdot s_j) / s_j$              |
| Tune                      | grid-search $\alpha \in [0, 1]$ , minimize $E$    |

## Appendix A — Reference implementation

Every number above is produced by `m02_t03_awq_engine.py`. Run it to reproduce the baseline, the scale sweep, and the 89.6% error reduction; change  $W$  or  $x_{\text{mag}}$  to move the salient channel.

```

1  """
2  Reference implementation for: Module 02 - Topic 3
3  "AWQ - Activation-aware Weight Quantization. Protect the salient channels."
4
5  We take a TINY layer (1 output row, 4 input channels). One channel is SALIENT:
6  its mean activation magnitude is ~50x the others, so a small weight error there
7  becomes a large OUTPUT error. AWQ scales that channel's weight UP by a factor s
8  (and divides its activation by the same s, via (W*s)(X/s)=WX) before quantizing,
9  which protects it from rounding. We grid-search s and measure the
10 activation-weighted error sum( |W - W_hat| * mean|X| ).
11
12 Every number printed here is transcribed (rounded) into the worked-by-hand PDF.
13 """
14
15 import numpy as np
16
17 np.set_printoptions(precision=4, suppress=True)
18
19 BITS = 4
20 QMAX = 2 ** (BITS - 1) - 1          # symmetric INT4 -> 7
21
22
23 # -----
24 # 0. The tiny layer. Channel 0 is SALIENT (huge activation magnitude).
25 # -----
26 W = np.array([0.18, 0.92, -0.55, 0.30])    # one output row, 4 input weights
27 x_mag = np.array([5.00, 0.10, 0.12, 0.08]) # mean |activation| per channel
28 print("== 0. Tiny layer ==")
29 print("W      =", W)
30 print("x_mag =", x_mag, " <- channel 0 is salient (50x the rest)")
31 print("absmax(W) =", np.abs(W).max(), " (the big weight 0.92 sets the per-tensor scale)")
32
33
34 def quantize(Wrow):
35     """Symmetric per-tensor INT4: one scale = absmax/QMAX for the whole row."""
36     s = np.abs(Wrow).max() / QMAX
37     q = np.clip(np.round(Wrow / s), -QMAX - 1, QMAX)
38     return s * q, s
39
40
41 def act_weighted_error(W, W_hat, x_mag):
42     """Error that actually reaches the output: |dW| weighted by activation size."""
43     return float((np.abs(W - W_hat) * x_mag).sum())
44

```

```

45
46 # -----
47 # 1. Baseline: quantize W directly, no scaling. The salient channel's
48 #   small weight (0.18) gets a coarse share of the grid (absmax=0.92).
49 # -----
50 W_base, s_base = quantize(W)
51 err_base = act_weighted_error(W, W_base, x_mag)
52 print("\n== 1. Baseline RTN W4 (no scaling) ==")
53 print(f"per-tensor scale = absmax/7 = {s_base:.5f}")
54 print("W_hat  =", W_base)
55 print("|dW|   =", np.abs(W - W_base))
56 print(f"activation-weighted error = {err_base:.6f}")
57 print(f"  channel 0 alone contributes {np.abs(W[0]-W_base[0])*x_mag[0]:.4f}"
58       f" ({np.abs(W[0]-W_base[0])*x_mag[0]/err_base*100:.0f}% of total)")
59
60 # -----
61 # 2. The AWQ identity:  scaling channel 0 of W UP by s and dividing the
62 #   activation channel 0 DOWN by s leaves the output W X unchanged.
63 # -----
64 print("\n== 2. AWQ scaling identity ==")
65 print("  (W*s) @ (X/s) = W @ X exactly. We use it ONLY on the salient channel:")
66 print("  scale W[0] up by s, fold the /s into the activation (or prior layer).")
67
68 # -----
69 # 3. Grid-search the salient-channel scale s in a sensible range.
70 #   Scale W[0] *= s, requantize the row, then divide W_hat[0] back by s.
71 # -----
72 print("\n== 3. Grid search over salient-channel scale s ==")
73 grid = [1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0, 5.0]
74 best = (np.inf, None, None)
75 for s in grid:
76     Ws = W.copy()
77     Ws[0] *= s                                # inflate the salient weight
78     Wq, _ = quantize(Ws)
79     W_hat = Wq.copy()
80     W_hat[0] /= s                             # fold activation /s back out
81     err = act_weighted_error(W, W_hat, x_mag)
82     flag = ""
83     if err < best[0]:
84         best = (err, s, W_hat)
85     print(f"  s={s:.1f}  W0_scaled={Ws[0]:+.3f}  W_hat={W_hat}  err={err:.6f}")
86
87 err_awq, s_star, W_hat_star = best
88 print("\n== 4. Best AWQ result ==")
89 print(f"  best salient scale s = {s_star:.1f}")
90 print(f"  W_hat   = {W_hat_star}")
91 print(f"  |dW| on salient channel 0 = {abs(W[0]-W_hat_star[0]):.5f}"
92       f"    (was {abs(W[0]-W_base[0]):.5f})")
93 print(f"  activation-weighted error = {err_awq:.6f}  (baseline {err_base:.6f})")
94 print(f"  AWQ reduces activation-weighted error by {(1-err_awq/err_base)*100:.1f}%")
95
96 # -----
97 # 5. The per-channel formula used at scale:  $s_j = x_{\text{mag}}^{\alpha} / w_{\text{mag}}^{(1-\alpha)}$ .
98 #   Show the value it suggests for the salient channel at  $\alpha=0.5$ .
99 # -----
100 print("\n== 5. The production scale formula   $s_j = \max|X_j|^a / \max|W_j|^{(1-a)}$  ==")
101 w_mag = np.abs(W)
102 for alpha in (0.0, 0.5, 1.0):
103     s = (x_mag ** alpha) / (w_mag ** (1 - alpha))
104     s = s / s.mean()                        # normalize, mean(s)=1
105     print(f"  alpha={alpha:.1f}:  s = {s}  (salient channel gets s0={s[0]:.2f})")
106 print("  larger alpha => protect salient channels harder; tune by grid search.")

```

— END OF DOCUMENT —